

Testy bez chmury: Q11 z modułem Shelly

Poziom B: Wi-Fi i internet są, chmura Shelly wyłączona w module. Testy B1–B12 · AMPERE POINT · 28 września 2026 · oprac. Dawid Cekała

W skrócie

- **Poziom B** to obecny stan DevKitu: chmura wyłączona od 25.09, sieć biura z internetem. Nic nie trzeba przestawiać.
- **Większość podstaw już sprawdziliśmy** (rozdział 1): sterowanie przez API, MQTT, OCPP przez most, godzina z internetu.
- **Zostało:**
 - czy da się sterować prosto, bez programowania: aplikacja Shelly, strona WWW i Home Assistant (B1–B3);
 - co moduł robi sam: harmonogram, webhook i pamięć (B4–B6);
 - jak działa API i gdzie są jego granice (B7–B12).
- **Ryzyko:** małe. Jedynie hasło do API (B11) może odciąć nasze narzędzia, więc robimy je na końcu programu.
- Poziomy łączności, pojęcia i porównanie z Tuya są w dokumencie 00, w tabeli 3.1.

1 Co już sprawdziliśmy na tym poziomie

Funkcja	Wynik	Kiedy	Szczegóły
Limit prądu i włącznik przez HTTP i WebSocket	✓	25.09	sterownik potwierdza; najpierw odsyła starą wartość, po około 1 s nową
MQTT z własnym brokerem	✓	25.09	limit 10 A przez MQTT potwierdzony przez sterownik w tej samej sekundzie
OCPP przez most	🟡	25.09	rejestracja, stan, znak życia, pomiary na żądanie, limit 10 i 12 A, zdalny start; pełna transakcja czeka na obciążenie
Godzina z publicznego serwera czasu	✓	25.09	chmura wyłączona o 11:48; o 11:51 sterownik dostał poprawną godzinę
Ikona sieci na wyświetlaczu	🟡	25.09	moduł wysyła stan „4” (połączony z chmurą), choć chmura jest wyłączona
Dziennik pracy modułu przez sieć	✓	24–25.09	cała diagnostyka bez kabla USB
Aplikacja Shelly przy włączonej chmurze	✓	25.09	10:52–10:54: włącznik 4 razy, limit 6 razy; każde polecenie przyszło przez chmurę Shelly (w logu „via SHC”). Po wyłączeniu chmury o 11:48 aplikacja nie wysłała ani jednego polecenia; wszystkie późniejsze pochodzą z naszych narzędzi na laptopie
Aktualizacja z portalu przez Bluetooth	🟡	24.09	działa; przesyłanie potrafi się przerwać; wgranie konfiguracji kasuje Wi-Fi modułu

2 Przygotowanie

- DevKit w sieci biura pod adresem 192.168.0.238, chmura wyłączona.

- **Serwer czasu: publiczny** (`time.cloudflare.com`), tak jak u klienta z internetem. Od 25.09 moduł jest przedstawiony na nasz lokalny serwer 192.168.0.57; przed testami B wracamy do publicznego. Lokalny serwer jest potrzebny dopiero w dokumencie 02 i w testach D5–D6.
- Broker MQTT może działać; testy B go nie używają, a MQTT na tym poziomie sprawdziliśmy 25.09.
- Monitor logu uruchomiony.
- Symulator auta podłączony (B5, B7).
- Na laptopie Python z bibliotekami do prostego serwera HTTP i WebSocket (B5, B8).

3 Testy

3.1 B1. Aplikacja Shelly w tej samej sieci

Cel. Sprawdzić, czy zwykły użytkownik steruje ładowarką z aplikacji, gdy chmura modułu jest wyłączona. 25.09 aplikacja sterowała wyłącznie przez chmurę. Źródło polecenia w logu rozstrzygnie, czy bez chmury przejdzie na sieć lokalną: będzie to adres telefonu w sieci biura.

Krok	Co robimy	Wynik oczekiwany
B1a	Telefon w sieci biura, aplikacja Shelly, otwieramy DevKit	zapisujemy, czy urządzenie jest dostępne, czy „offline”
B1b	Zmiana limitu prądu i włącznika z aplikacji	jeśli działa: w logu modułu widać źródło polecenia z sieci lokalnej, sterownik potwierdza
B1c	Ten sam telefon na danych komórkowych	oczekujemy, że nie działa, bo zdalny dostęp idzie przez chmurę; potwierdzamy

3.2 B2. Strona WWW modułu na produkcie Q11

Cel. Strona modułu to droga bez aplikacji i bez konta. Działała 24.09 na symulatorze; teraz sprawdzamy ją na produkcie Q11.

Krok	Co robimy	Wynik oczekiwany
B2a	Laptop: przeglądarka, adres <code>http://192.168.0.238</code>	strona się otwiera, widać pola ładowarki
B2b	Zmiana limitu i włącznika ze strony	sterownik potwierdza w 1 s
B2c	Telefon w sieci biura: to samo	działa na telefonie; zapisujemy, czy jest czytelna
B2d	Czy strona pyta o hasło	dziś nie pyta; wniosek do analizy GAP: każdy w sieci może sterować ładowarką, dopóki nie ma hasła (B11)

3.3 B3. Home Assistant z oficjalną integracją Shelly

Cel. Tuya w Home Assistant wymaga klucza lokalnego i naszego profilu. Sprawdzić, czy Shelly wchodzi do Home Assistant bez tego.

Krok	Co robimy	Wynik oczekiwany
B3a	W naszym Home Assistant: dodanie urządzenia Shelly pod adresem 192.168.0.238	Home Assistant wykrywa moduł
B3b	Przegląd encji	zapisujemy, które pola ładowarki są widoczne: stan, limit, włącznik, fazy, energia i czas sesji
B3c	Zmiana limitu i włącznika z Home Assistant	sterownik potwierdza
B3d	Po teście	urządzenie zostaje albo je usuwamy, decyzja użytkownika. Dodanie zmienia konfigurację naszego Home Assistant

3.4 B4. Harmonogram w module

Cel. Harmonogram zapisany w module ma działać bez chmury i przetrwać restart.

Krok	Co robimy	Wynik oczekiwany
B4a	Dwa zadania: za 2 min limit 6 A, za 4 min 16 A	oba odpalają o czasie, sterownik potwierdza
B4b	Restart modułu poleceniem, bez zaniku zasilania; nowe zadanie za 2 min	zadania są po restarcie i działają

3.5 B5. Webhook

Cel. Moduł ma sam zawiadamiać nasz system o zdarzeniach. Przykład: „zaczęło się ładowanie” trafia do programu na laptopie.

Krok	Co robimy	Wynik oczekiwany
B5a	Lista zdarzeń, na które moduł umie wywołać webhook (polecenie, które nic nie zmienia)	wiemy, czy zmiana pól ładowarki (stan, limit, włącznik) jest zdarzeniem, czy są tylko zdarzenia systemowe
B5b	Nasłuch HTTP na laptopie; webhook na zmianę stanu ładowania; symulator: wolne → podłączone → łąduje	wywołanie przy każdej zmianie, z treścią zdarzenia. Jeśli B5a pokaże brak takiego zdarzenia: X i powód
B5c	Restart modułu, powtórka B5b	webhook zostaje i działa

3.6 B6. Pamięć klucz-wartość

Cel. Pamięć, w której skrypt mógłby trzymać dane sesji, np. energię i czas, żeby nie ginęły przy restarcie.

Krok	Co robimy	Wynik oczekiwany
B6a	Zapis wartości testowej	odczyt daje to samo
B6b	Restart modułu	wartość zostaje; zanik zasilania sprawdza C8

3.7 B7. API: wartości spoza zakresu

Cel. Ustalić, kto pilnuje granic: pole modułu, skrypt czy sterownik. Definicja Tuya dopuszcza limit 6–32 A, a Q11 ma 16 A. Z symulatorem bez obciążenia to bezpieczne.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B7a	Limit 5 A, 17 A, 20 A, 32 A	odpowiedź API (błąd czy zgoda) i czy coś poszło do sterownika; co sterownik odesłał
B7b	Limit 10,5 A i tekst zamiast liczby	to samo
B7c	Nieznane pole i nieznane polecenie	kod błędu API

3.8 B8. API: wychodzący WebSocket

Cel. Moduł sam łączy się z naszym serwerem. Nie trzeba wtedy otwierać portów w sieci klienta, a to podstawa tłumacza OCPP na serwerze AmperePoint.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B8a	Serwer testowy na laptopie; w module adres tego serwera	moduł łączy się sam; zapisujemy, co wysyła po połączeniu i przy zmianach
B8b	Serwer odsyła polecenie: limit 10 A	polecenie dociera, sterownik potwierdza
B8c	Wyłączenie serwera na 1 min	czy i po ilu sekundach moduł łączy się ponownie

3.9 B9. API: połączenia naraz

Cel. Most OCPP, monitor, strona WWW, Home Assistant i aplikacja mogą łączyć się jednocześnie. Trzeba znać granicę.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B9a	Most, monitor i strona WWW otwarte; dokładamy kolejne połączenia WebSocket z laptopa	przy którym połączeniu moduł odmawia albo zwalnia; czy zrywa któreś z istniejących

3.10 B10. API: kopia konfiguracji

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B10a	Utworzenie i pobranie kopii konfiguracji na laptop	co zawiera. Jeśli hasło Wi-Fi, plik nie może trafić do repozytorium

3.11 B11. API: hasło

Cel. Zabezpieczyć moduł w sieci klienta i ustalić, które drogi hasło obejmuje.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B11a	Ustawienie hasła	—
B11b	Próba bez hasła i z hasłem: HTTP, WebSocket, strona WWW, Bluetooth, MQTT	które drogi wymagają hasła
B11c	Zdjęcie hasła	narzędzia działają jak przedtem

Ryzyko: średnie. Most OCPP i monitor przestaną działać, dopóki nie dopiszemy logowania. Hasło zapisujemy w pliku projektu, poza repozytorium, bo zgubione oznacza reset fabryczny.

3.12 B12. Kanał UDP, opcjonalnie

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
B12a	Włączenie kanału UDP, polecenie odczytu stanu z laptopa	czy działa i do czego mógłby służyć

4 Kolejność i czas

Kolejność	Testy	Czas
1	B1–B3	50 min
2	B4–B6	1 h
3	B7–B10, B12	1 h 30 min
4	B11, na końcu całego programu	20 min

Razem około 3 h 40 min. Miejsce tych testów w kolejności całego programu: dokument 00, rozdział 5.

5 Wyniki

Test	Wynik	Co zaobserwowano	Data
B1 aplikacja w tej samej sieci	✗ wstępnie, do powtórzenia	B1a: telefon w sieci biura, aplikacja pokazuje DevKit jako „offline”. W logu jedno połączenie z 192.168.0.146 o 07:12:06, otwarte i zamknięte bez polecenia. To adres tego telefonu (potwierdzony w B2c), czyli aplikacja zajrzała do modułu lokalnie, ale nic nie wysłała. B1b niewykonalne, bo urządzenie jest „offline”. B1c wynika z B1a. Wniosek wstępny: bez chmury aplikacja Shelly nie steruje, drogą dla użytkownika zostaje strona WWW (B2). Do powtórzenia po wgraniu v4, przy wyłączonej chmurze, z ekranem urządzenia otwartym w aplikacji	29.09 07:12
B1 powtórka (v4.2/v4.3)	✗	Aplikacja dalej „offline”. Telefon 192.168.0.146 łączył się z modułem 4 razy (09:44, 11:53, 13:01:08, 13:01:20). Za każdym razem krótkie zapytanie i zamknięcie po 3–6 s, bez polecenia i bez stałego połączenia; w logu wygląda to tak samo jak zapytanie o dane urządzenia (/shelly) z laptopa. Sieć działa: HA, strona WWW i laptop sterują modułem. Wniosek: aplikacja znajduje moduł w sieci, ale steruje przez chmurę. Rozstrzygnięcie próba kontrolna: chmura włączona na 2–3 min; jeśli aplikacja ożyje, przyczyną jest brak chmury	29.09 13:28
B1 próba kontrolna	✓ rozstrzygnięte	Chmura włączona 13:43–13:45 (bez restartu przy wyłączaniu). Aplikacja od razu „online”. Polecenia z aplikacji przysły przez serwer Shelly (SHC 34.40.19.158:6022, nadawca iotc_291_1-shelly-288-eu), nie przez sieć lokalną: limit prądu 6 → 12 A (sterownik potwierdził), wyłączenie ładowania. Wniosek: aplikacja Shelly steruje tylko przez chmurę; w sieci lokalnej jedynie sprawdza moduł. Przy wyłączeniu ładowania sterownik odpowiedział dwa razy „wł.”, a pole pokazuje „wył.”; moduł uznał odpowiedź za „bez zmian” i nie przekazał jej skryptowi (ten sam problem co odrzucony tryb w B3c)	29.09 13:45

Test	Wynik	Co zaobserwowano	Data
B2 strona WWW na Q11	✓	B2a ✓: strona pod 192.168.0.238 z laptopa otwiera się po około 5 s, tytuł „Ampere Point Q11 DevKit test”; widać włącznik, stan, fazy, moc, energię, czas sesji i limit z suwakiem 6–16 A. B2b ✓: limit 12 → 10 A ze strony, 07:13:25 zapis do sterownika, 07:13:27 potwierdzenie (najpierw stara wartość 12). Włącznik ze strony: 07:13:37 wyłączony, sterownik potwierdził i przeszedł w stan „czeka”; 07:15:34 włączony, znów „ładuje”. B2d: strona nie pyta o hasło. Usterki: kafelki faz pokazują prąd „N/A A”, choć moduł ma w polu faz prąd 0 i napięcie L1 227 V; czas sesji 1072 min to błąd skryptu (liczy z zegara). B2c ✓: telefon w sieci biura (192.168.0.146), strona czytelna. Z telefonu o 07:18:04 wyłączone ładowanie, o 07:18:12 limit 9 A, o 07:18:17 limit 14 A, o 07:18:19 włączone ładowanie; każde polecenie sterownik potwierdził w ciągu 1–2 s. Obserwacja: przy zmianie na 14 A moduł wysłał zapis do sterownika dwa razy w tej samej sekundzie, przy jednym poleceniu; nieszkodliwe, do wyjaśnienia	29.09 07:13–07:18
B3 Home Assistant	🟡 wstrzymany	B3a ✓: oficjalna integracja Shelly dodała „Q11 DevKit test” lokalnie; w logu połączenia przez WebSocket od biblioteki Home Assistant (aios-...), bez chmury. B3b 🟡: widoczne encje: energia x2, moc, moc, napięcie i prąd faz A–C (L1 227 V; prąd 0,00 A, czyli dane faz są poprawne, a „N/A” na stronie WWW to błąd strony). Encji sterowania (włącznik, limit, stan) jeszcze nie sprawdzaliśmy. Test wstrzymany do rozbudowy produktu o brakujące pola	29.09 07:20
B3b po v4.2	🟡	19 encji: 12 fazowych i 7 systemowych, żadnego z 14 pozostałych pól; „Wczytaj ponownie” nic nie zmienia. Przyczyna w kodzie integracji: pola usługi dostają encję tylko, gdy ich klucz jest na liście znanych kluczy. Fazy są na liście dla każdego urządzenia. Limit prądu i włącznik ładowania są na liście tylko dla ładowarki TopAC (kod produktu EVE01), a nasz kod to apq11dev. Stan, energia i czas sesji są na liście pod innymi nazwami (work_state, energy_charge, time_charge) niż w szablonie Shelly (mode, session_energy, session_duration). Pozostałe nasze pola mają klucze nieznane integracji	29.09 11:45

Test	Wynik	Co zaobserwowano	Data
B3 z łatką	✓	Kopia integracji Shelly w custom_components z dopisanym Q11 (folder DevKit\ha_integracja, opis w CZYTAJ.pdf). Po restarcie 33 encje: dochodzi 14, w tym limit prądu, wyłącznik ładowania, tryb ładowania, limit energii, okno od/do, stan ładowarki, sygnał pojazdu, energia i czas sesji, temperatura, usterki. W logu brak błędów	29.09 11:57
B3c	✓ z uwagami	Wszystkie polecenia z Home Assistant poszły lokalnie (WebSocket z 192.168.0.57) i sterownik je potwierdził: limit prądu 14 → 9 → 6 A, wyłącznik 6 razy, tryb, okno (1–1 odrzucone, 2–0 przyjęte), limit energii. Uwagi: (1) skrypt dwa razy wysłał starą wartość po nowej (limit energii 3 → 2, wyłącznik „wł.” bez polecenia), bo sterownik odpowiada z opóźnieniem i starą wartością; poprawka w skrypcie v4.3. (2) Odrzucenie trybu „do limitu energii” przy limicie 0 nie dotarło do pola: HA pokazywał tryb, którego sterownik nie przyjął. (3) Pole limitu energii w HA wysła polecenie po każdym kliknięciu strzałki: 24 polecenia w 9 s	29.09 12:04
B3c po v4.3.1	✓	Skrypt v4.3.1 wgrany 13:25 (usługa 13109-3815). Próba przez API modułu, tak jak wysła HA: seria 8 → 6 w 0,3 s (powrót do wartości sterownika) — nic nie poszło; seria 7 → 9 → 7 w 0,6 s — jeden zapis 7, sterownik najpierw odesłał starą 6, potem 7, bez powtórnego zapisu; powrót do 6 — jeden zapis. Po starcie modułu brak komunikatów (v4.3 przy każdym starcie odrzucała jedną zmianę trybu)	29.09 13:27
B4 harmonogram w module	✓	B4a ✓: dwa zadania (Schedule.Create, polecenie Number.Set limitu) odpaliły o 13:54:00 i 13:56:00 czasu modułu (czas z internetu, strefa Europe/Warsaw); skrypt wysłał po 1 s, sterownik potwierdził 6 A i 16 A. B4b ✓: po restarcie poleceniem (13:56:45) oba zadania były na liście, nowe zadanie odpaliło o 13:59:00, sterownik potwierdził 12 A. Zmiana z harmonogramu przychodzi do skryptu bez pola źródła, skrypt traktuje ją jak zmianę z zewnątrz. Zadania testowe usunięte (codzienne)	29.09 13:59

Test	Wynik	Co zaobserwowano	Data
B5 webhook	✓ (B5b niżej)	B5a ✓: Webhook.ListSupported ma zdarzenia pól: number.change, enum.change, boolean.change, text.change, object.change, więc zmiana stanu, limitu i włącznika może wywołać webhook. B5b: webhooks założone (stan z wartością w adresie, start ładowania z warunkiem <code>ev.value == "charger_charging"</code> , limit, włącznik); odbiornik w kontenerze Docker na laptopie (port 8099; zaporą Windows blokuje zwykłe programy). Na zmianę limitu wywołania dochodzą w ok. 1 s z wartością (<code>/limit?v=11</code>). Przejścia stanu z testera: do zrobienia. B5c ✓ dla limitu: webhooks są po restarcie i działają	29.09 14:05
B5b z testerem	✓	30.09 12:41, z opóźnieniem ustawionym w menu ładowarki. Webhooks (czas laptopa): <code>/stan?v=charger_charging</code> 12:41:12.8, <code>/ladowanie_start</code> (z warunkiem) 12:41:13.9, <code>/stan?v=charger_wait</code> 12:41:15.9, <code>/wlacznik?v=false</code> 12:41:38.8, <code>/stan?v=charger_free</code> 12:41:46.1. Każda zmiana stanu dała wywołanie w około 1 s, warunek na start ładowania działa. Stanu „podłączone” nie było: sterownik przeszedł od razu do „ładuje”. Sygnał z auta przełączany 9 V ↔ 6 V (12:41:28–43) nie zmienił stanu z „czeka”, co pasuje do działającego opóźnienia z menu	30.09 12:42
B6 pamięć klucz-wartość	✓	B6a: <code>KVS.Set/KVS.Get</code> dają to samo. B6b: po restarcie poleceniem wartość zostaje. Zanik zasilania: C8	29.09 14:05
B7 wartości spoza zakresu	✓	Granice pilnuje pole modułu: 5, 17, 20, 32 A → błąd -103 „out of bounds”, nic do sterownika. Tekst „abc” → -103; tekst „12” przyjęty jako liczba. 10,5 A przyjęte (moduł nie pilnuje kroku), skrypt wysłał 11 A, sterownik potwierdził. Nieznane pole → -105, nieznane polecenie → 404, opcja spoza listy → -103, zapis pola tylko do odczytu → -107 „Permission denied”	29.09 14:06

Test	Wynik	Co zaobserwowano	Data
B8 wychodzący WebSocket	✓	Serwer testowy w kontenerze na laptopie (port 8098). B8a: moduł połączył się sam 25 s po restarcie, przysłał pełny stan, potem każdą zmianę pola. B8b: polecenie z serwera „limit 10 A” → odpowiedź w 150 ms, zapis do sterownika. B8c: po zniknięciu serwera ponowienia po 30 s, potem co 60 s; po powrocie serwera połączenie po 28 s. Połączenie wychodzące wyłączone po teście	29.09 14:09
B9 połączenia naraz	✓ z uwagą	Poza HA, monitorem, stroną WWW i połączeniem wychodzącym moduł przyjął 6 kolejnych połączeń WebSocket; siódmego nie (brak odpowiedzi HTTP). Czasy odpowiedzi 110–170 ms. W chwili odmowy moduł zamknął połączenie Home Assistant, a HA połączył się ponownie: przy komplecie połączeń nowy klient może wyrzucić istniejącego	29.09 14:10
B10 kopia konfiguracji	✓ z uwagą	Sys.CreateBackup + restart, potem Sys.DownloadBackup z parametrem offset (po 1024 B; bez niego wciąż ten sam kawałek). ZIP 9 kB, niezaszyfrowany : konfiguracja z hasłem Wi-Fi i tokenem chmury, pamięć klucz-wartość, webhooki, harmonogram, stan pól. Bez plików usługi (skrypt, ekran). Plik w DevKit\kopie_konfiguracji z ostrzeżeniem, poza repozytorium	29.09 14:12
B11 hasło	czeka na Dawida	Ustawienie hasła to wpisanie hasła; robi to Dawid	
B13 MQTT z brokerem w internecie (dodatkowy)	✓	01.10 11:43–11:48. Moduł z internetem (adres automatyczny), broker publiczny test.mosquitto.org:1883, losowy przedrostek tematów. Moduł połączył się z brokerem i utrzymał połączenie; jego wiadomości (online, stan, zdarzenia) dochodziły do laptopa przez internet. Polecenie limitu 10 A z laptopa przez broker: odpowiedź modułu po około 1,1 s, sterownik potwierdził; powrót do 12 A tak samo. Broker publiczny zrywał co drugie połączenie laptopa przy łączeniu (klient bez identyfikatora odrzucany, potem losowo); połączenie modułu stabilne. Po teście broker z powrotem na laptopie. Uwaga: w tym czasie chmura modułu była włączona (włączyło ją ponowne dodanie w aplikacji 30.09)	01.10 11:48

Test	Wynik	Co zaobserwowano	Data
B12 kanał UDP	✓	Port 1010: odczyt 40–60 ms, zapis 135 ms, polecenie dochodzi do sterownika. Tylko pytanie–odpowiedź, bez zdarzeń. Bez hasła, więc po teście wyłączony	29.09 14:13